Problem 2: Weak Scalability Study

The weak-scaling speedup and efficiency were computed as

$$S_p = \frac{T_1(np)}{T_p}, \qquad E_p = \frac{S_p}{p}.$$

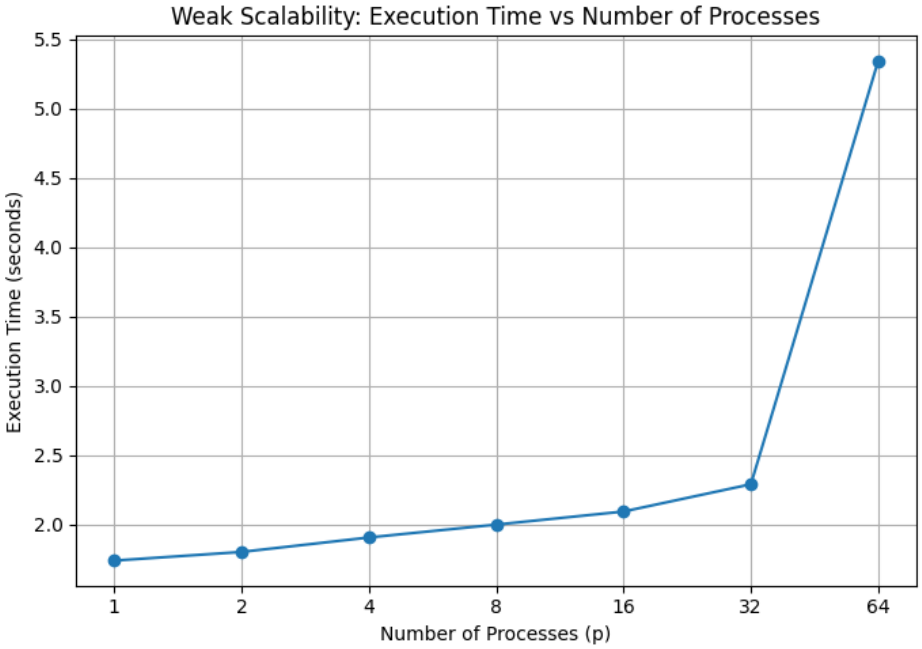

Figure 1: Weak scalability: execution time versus number of processes.

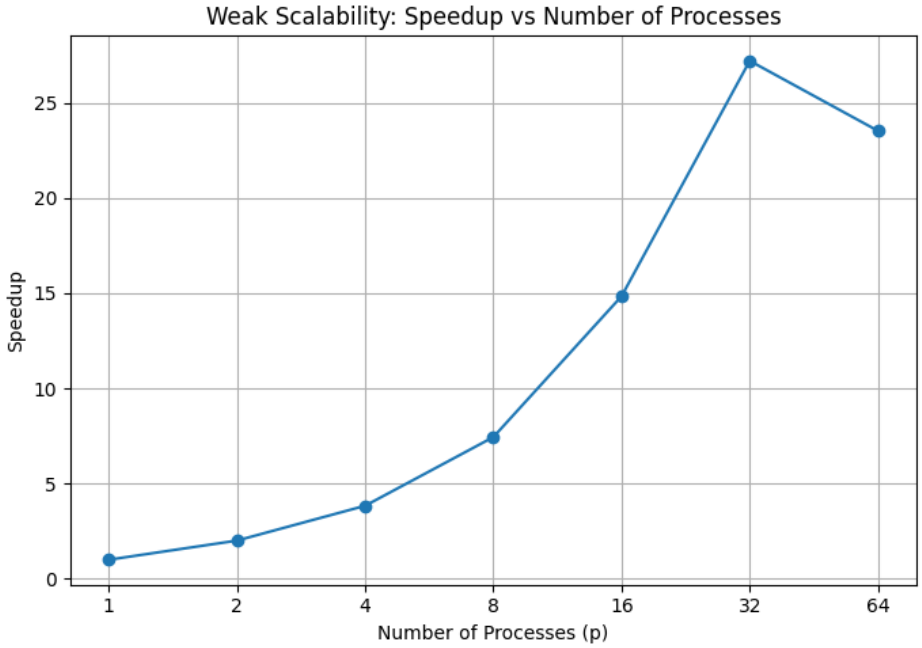

Figure 2: Weak scalability: speedup versus number of processes.

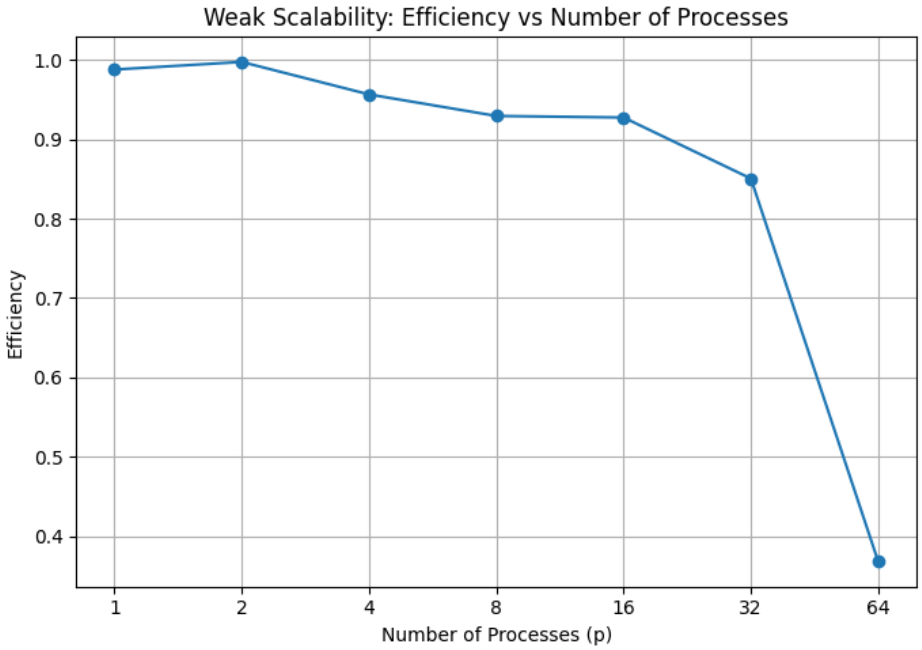

Figure 3: Weak scalability: efficiency versus number of processes.

Problem 3: Strong Scalability Study

The strong-scaling speedup and efficiency were computed as

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p}.$$

For larger process counts, the results become less regular. In particular, the execution time at $p = 32$ is higher than expected, while the time at $p = 64$ decreases again. This is likely due to communication overhead, synchronization cost, process placement, and system noise. As the local workload becomes smaller at higher process counts, these overheads account for a larger fraction of the total runtime.

Overall, the implementation shows good strong scalability up to moderate process counts, while the irregular behavior at $p = 32$ and $p = 64$ suggests that repeated runs and averaged timings would provide more stable results.

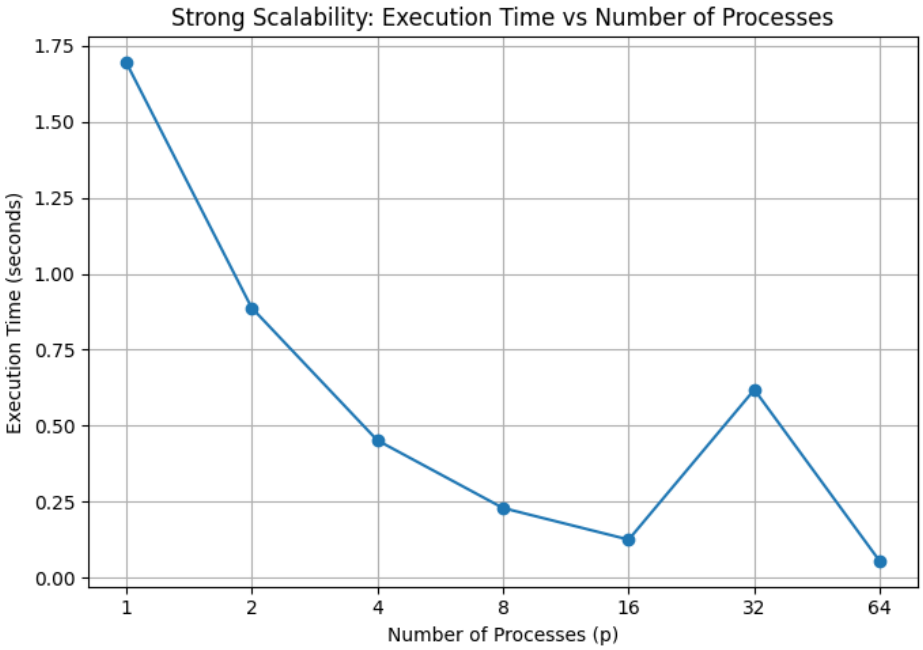

Figure 4: Strong scalability: execution time versus number of processes.

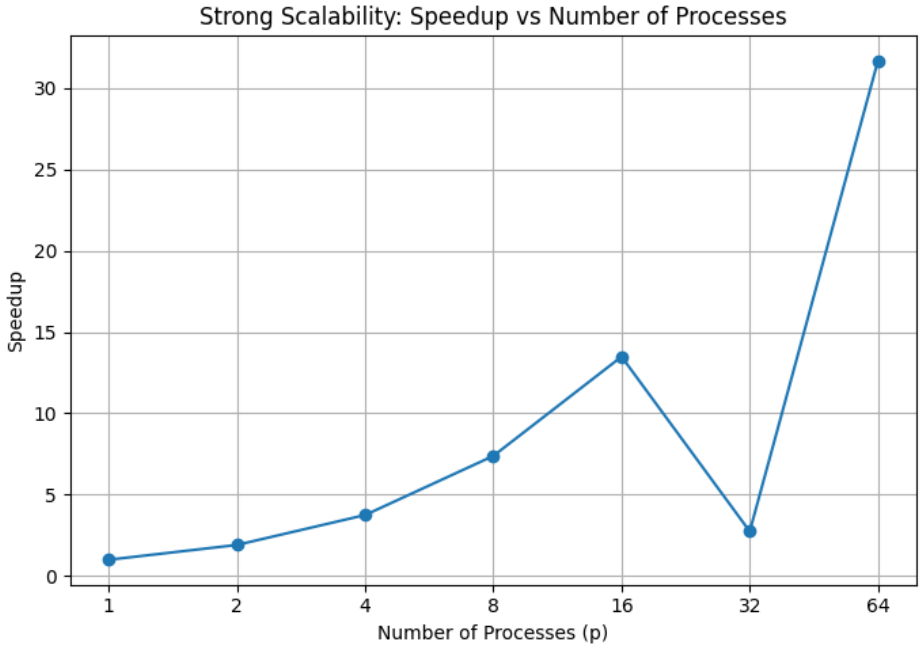

Figure 5: Strong scalability: speedup versus number of processes.

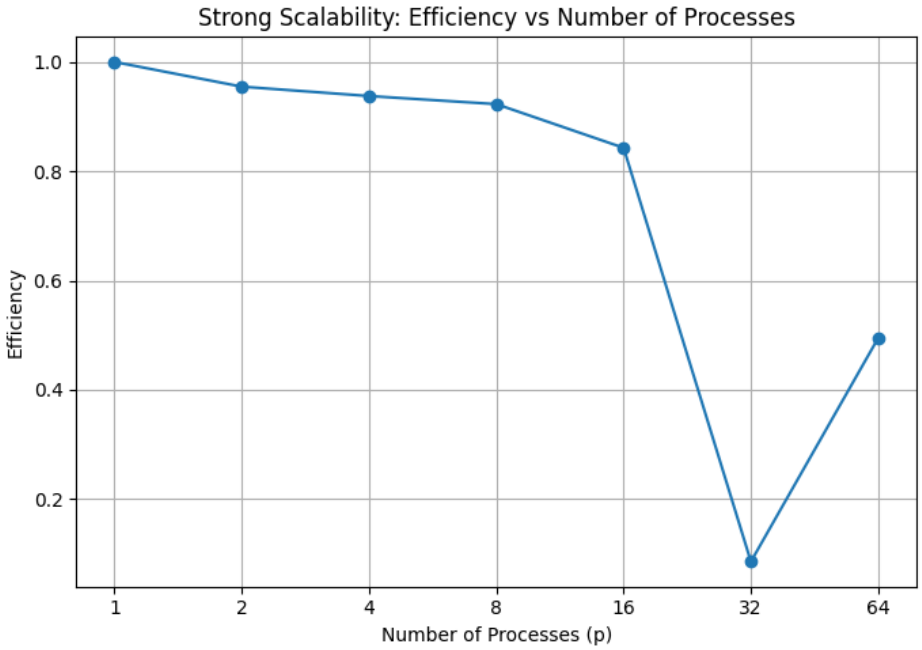

Figure 6: Strong scalability: efficiency versus number of processes.